

Separación del Código & Distribución del Procesamiento de las Aplicaciones

Lenguajes para aplicaciones comerciales

MSI. Álvaro Mena Monge

I-2026



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes Regionales

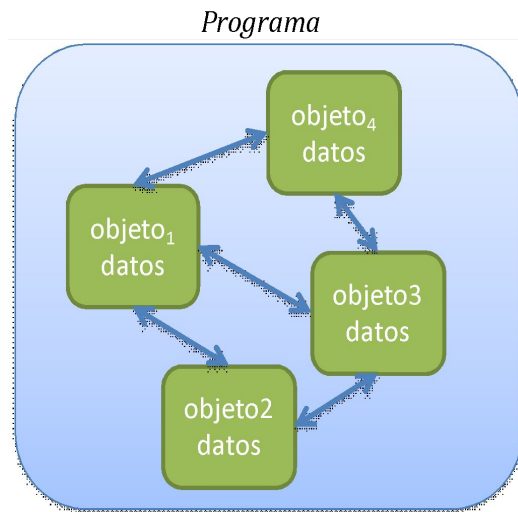
Agenda

- ¿Por qué emplear capas en la codificación?
- La arquitectura multicapas lógica (n-layer) y sus beneficios
- Arquitectura multicapas física (n-tier) y sus beneficios
- Implementación de funcionalidad en Spring en 3-capas
- Evitar aplicaciones monolíticas. Pensar en microservicios



¿Por qué emplear capas en la codificación?

Las aplicaciones de software están integradas por objetos que interactúan entre ellos para lograr ciertas tareas



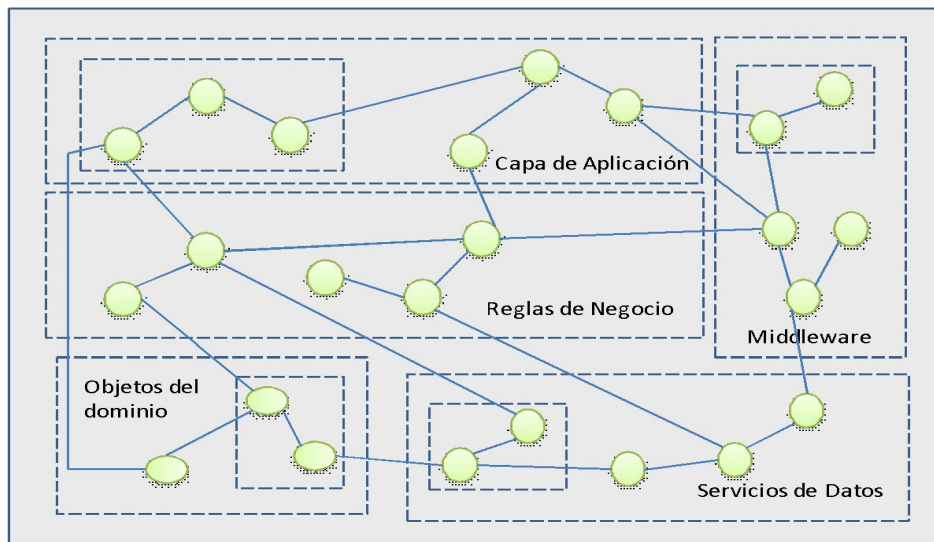
¿Por qué emplear capas en la codificación?

- Los objetos de una aplicación están diseñados de forma que se especializan en diversas tareas:
 - Mantener **el estado de la aplicación** (la información del proceso de negocio) en tiempo de ejecución.
 - Permitir **la interacción de la persona usuaria** a través de interfaces gráficas.
 - **Filtrar las entradas de las personas usuarias** para que respeten las reglas del negocio.
 - **Garantizar la persistencia de la información** suministrada por la persona usuaria y/o calcularla a partir de procesos generados por la aplicación



¿Por qué emplear capas en la codificación?

¿Será posible encontrar una forma más conveniente de organizar estos objetos colaborativos?



Arquitectura multicapas lógica

N-layer architecture



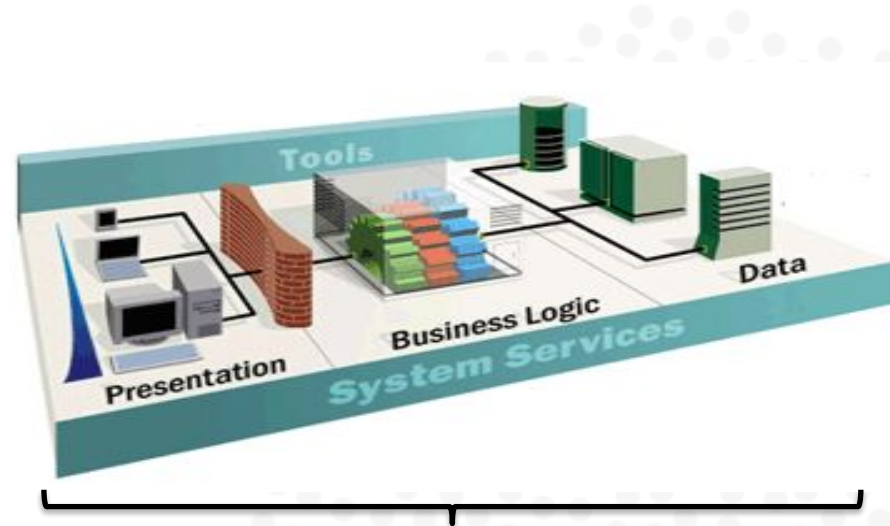
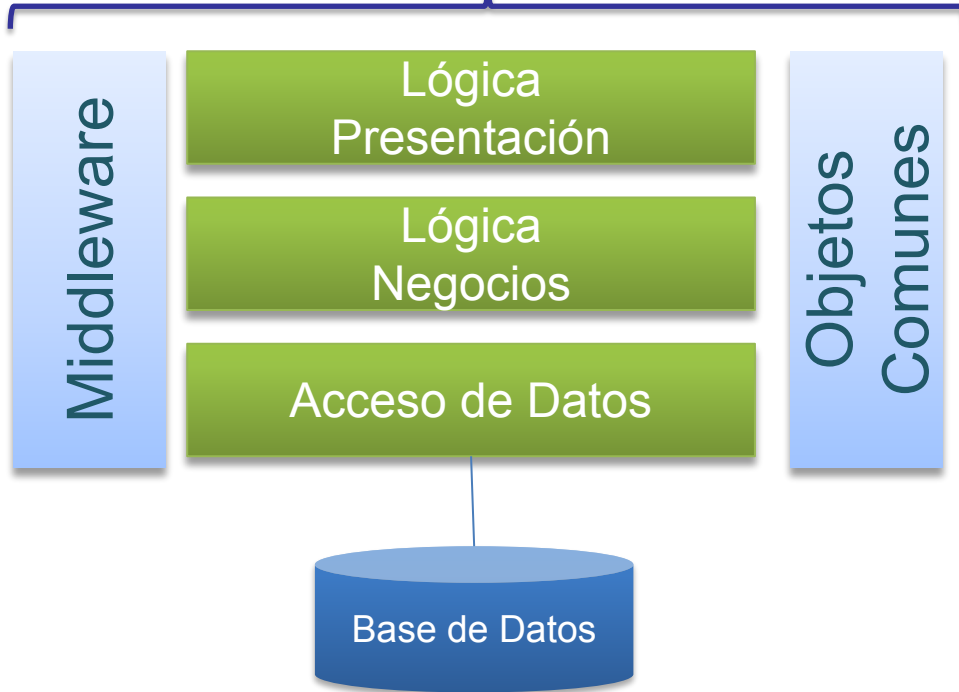
Arquitectura multicapas lógica

- **Arquitectura multicapas:** “un sistema de software que es segregado en secciones separadas referidas como capas (layers)” Sarknas (2006)
- Referido también como *arquitectura n-capas*
- Dicha partición se realiza a nivel de **la lógica de implementación (código)**



Arquitectura multicapas: lógica versus física

Separación lógica de las capas



Distribución física de las capas



Arquitectura multicapas lógica

- Capa de Presentación
 - Presenta la interfaz de usuario (UI) para la aplicación
 - Despliega información y/o recolecta las entradas ingresadas por la persona usuaria
 - Envía solicitudes a la siguiente capa (lógica de negocios)
- Capa de Lógica de Negocios(1)
 - Incorpora las reglas de negocio de la aplicación
 - Recibe solicitudes de la capa presentación, las evalúa contra las reglas de negocio y las pasa a la capa de datos



Arquitectura multicapas lógica

- Capa de Lógica de Negocios(2)

- También recibe datos desde la capa de datos y las entrega a la capa de presentación

- Capa de Servicio de Datos

- Se comunica directamente con el almacén de datos (Sql Server, Oracle, XML, hoja electrónica, servicios web)
- Sus objetos se especializan en recuperar, insertar, borrar y actualizar datos en el almacén de datos



Arquitectura multicapas lógica

■ Capa de Objetos Comunes

- Representan el modelo de dominio del problema siendo resuelto
- Responsable de representar los datos y las reglas de negocio de la aplicación
- Usualmente son objetos persistentes y por ende mantienen el estado de la aplicación
- Algunos ejemplos: Cuenta, Estudiante, Libro, Venta, DetalleVenta

■ Middleware

- Representa la API ofrecida por la plataforma en que se desarrolla
- Muy valiosa porque provee una línea base para el desarrollo



Beneficios de la arquitectura n-capas



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes Regionales

Beneficios de la Arquitectura N-Capas lógica

- *Ayuda a reducir la complejidad*
 - Separa la implementación en piezas más manejables
 - Se reduce el “código espagueti”
- *Favorece la **reutilización***
 - Cada capa es construida de forma independiente
 - Las capas superiores dependen de las inferiores
 - La misma lógica puede ser utilizada por muchos clientes o aplicaciones



Beneficios de la Arquitectura N-Capas lógica

- *Flexibilidad*
 - Es posible realizar cambios en cada capa o integrar nuevos objetos sin mucho esfuerzo
- Mantenimiento y soporte más simples
 - Los cambios (mejora, corrección de un defecto, nuevo requerimiento) se focalizan sobre la capa respectiva
- *Reduce la distribución de actualizaciones de las aplicaciones*
 - La actualización se realiza en la capa respectiva tal que todas las aplicaciones quedan actualizadas automáticamente Ej.
Cambio en legislación



Beneficios de la Arquitectura N-Capas lógica

- *Reduce la distribución de actualizaciones de las aplicaciones*
 - La actualización se realiza en la capa respectiva tal que todas las aplicaciones quedan actualizadas automáticamente Ej. Cambio en legislación



Beneficios de la Arquitectura N-Capas lógica

- Posibilita el desarrollo en paralelo
 - Ej. Un desarrollador/a (o un equipo de ellos) puede trabajar en la capa de presentación, otro en la capa de lógica de negocios y así sucesivamente
- Robustez
 - La separación provee encapsulación de los distintas capas o componentes
 - Cada capa es tratada como una *caja negra*



Aquitectura multicapas física

N-tier architecture



Arquitectura N-Tier (física)

- Distribución de la ejecución de las capas de una aplicación en distintas máquinas o nodos de procesamiento
 - La presentación (UI) corre en la computadora del usuario (***aplicación cliente***)
 - La lógica de aplicación, de negocios y de servicios de datos corre en un ***servidor de aplicaciones***
 - Los datos son almacenados en un **servidor de base de datos**



Beneficios de la Arquitectura N-Tier (física)

■ *Escalabilidad(1)*

- Es una medida de cuánto cambia el performance (rendimiento) conforme más procesamiento es agregado a una aplicación
- Si bien las capas de una aplicación pueden residir en un mismo servidor, es posible dispersar las capas en servidores o contenedores adicionales
- Anticipa la posibilidad de que la aplicación deba brindar mayor rendimiento ante un crecimiento en la demanda

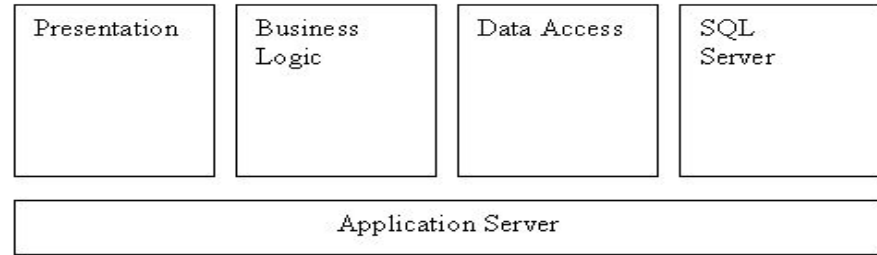


Beneficios de la Arquitectura N-Tier (física)

- *Escalabilidad(2)*

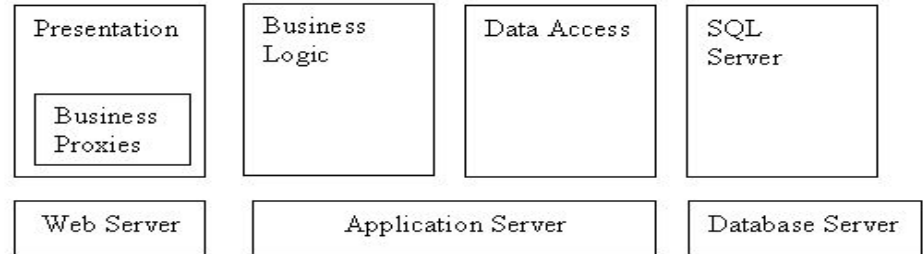
Capas instaladas en
un servidor de aplicación

Figure 1



Capas instaladas en
distintos servidores
de aplicación

Figure 2



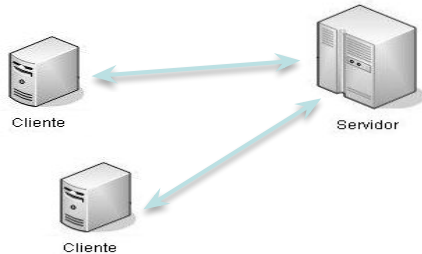
Beneficios de la Arquitectura N-Tier (física)

- Performance:
 - Velocidad con la cual la aplicación responde al usuario
- Tolerancia a fallas: dado a que se provee redundancia
- Seguridad: las capas (tiers) físicas pueden incrementar o decrementar el acceso físico a las máquinas en las cuales las aplicaciones son ejecutadas



Modelo Cliente-Servidor (físico)

- Modelo para computación distribuida que divide el procesamiento entre los clientes y los servidores
 - Los clientes solicitan servicios a los servidores
 - Los servidores proveen servicios
- Conceptos: fat clients, thin clients, rich clients

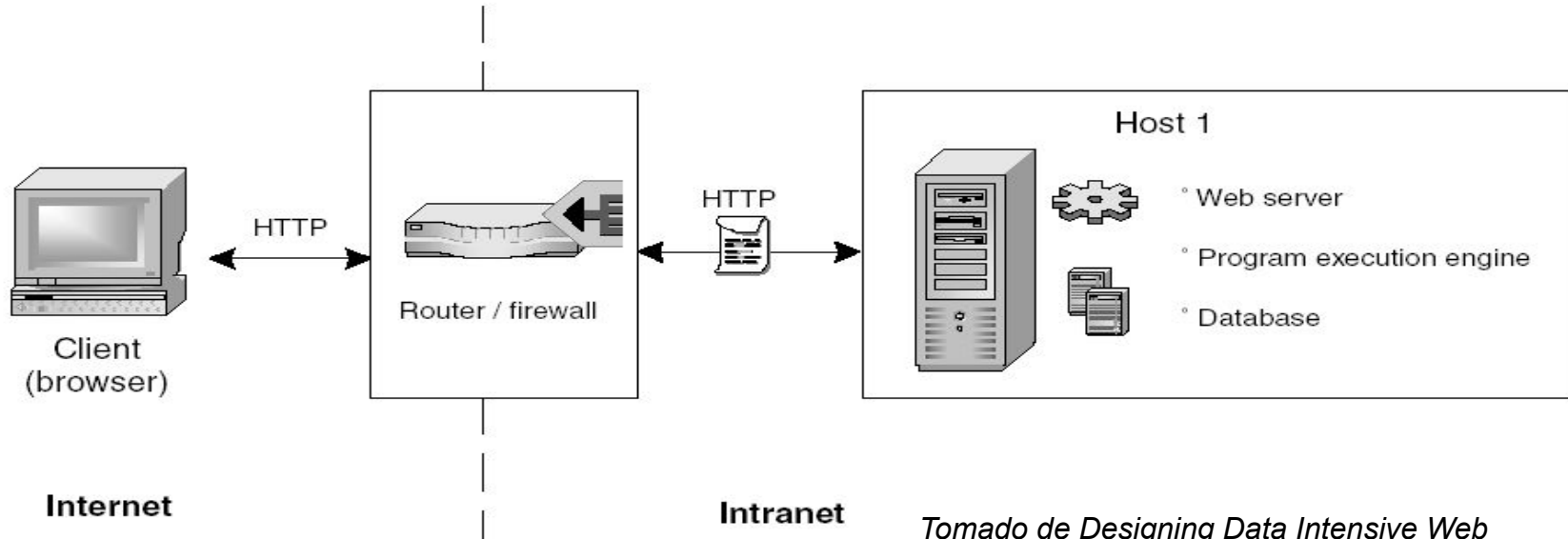


Las aplicaciones cliente-servidor originales corrían en estaciones de trabajo y accedían a los datos en el servidor de b.d.



Arquitectura N-Tier (física)

■ Modelo Cliente-Servidor

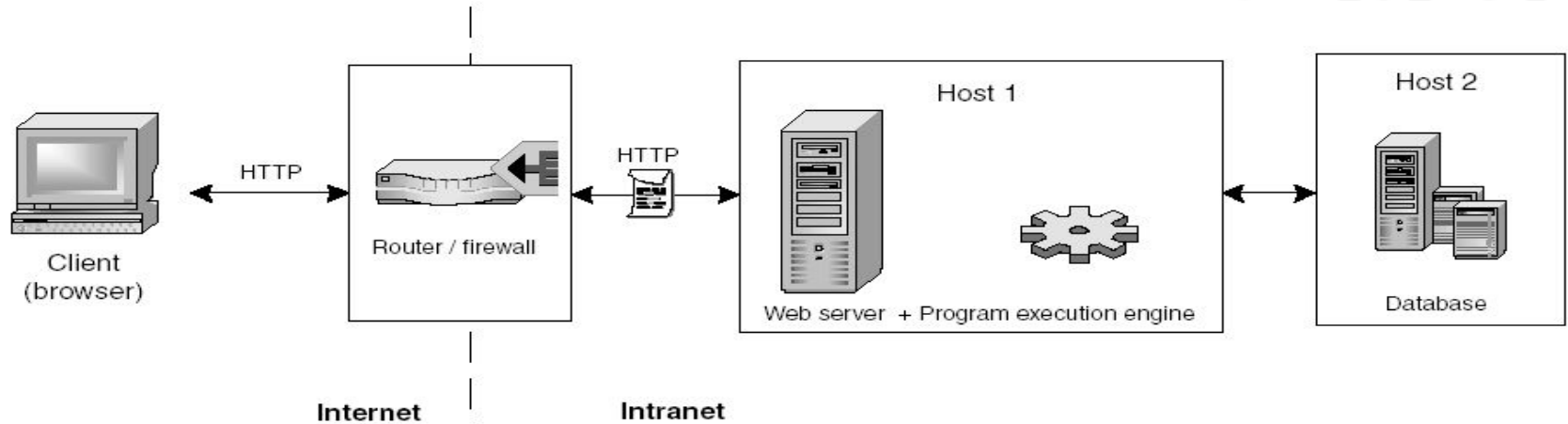


Tomado de Designing Data Intensive Web Applications. Stefano et al.



Arquitectura N-Tier (física)

■ Modelo Tres Capas

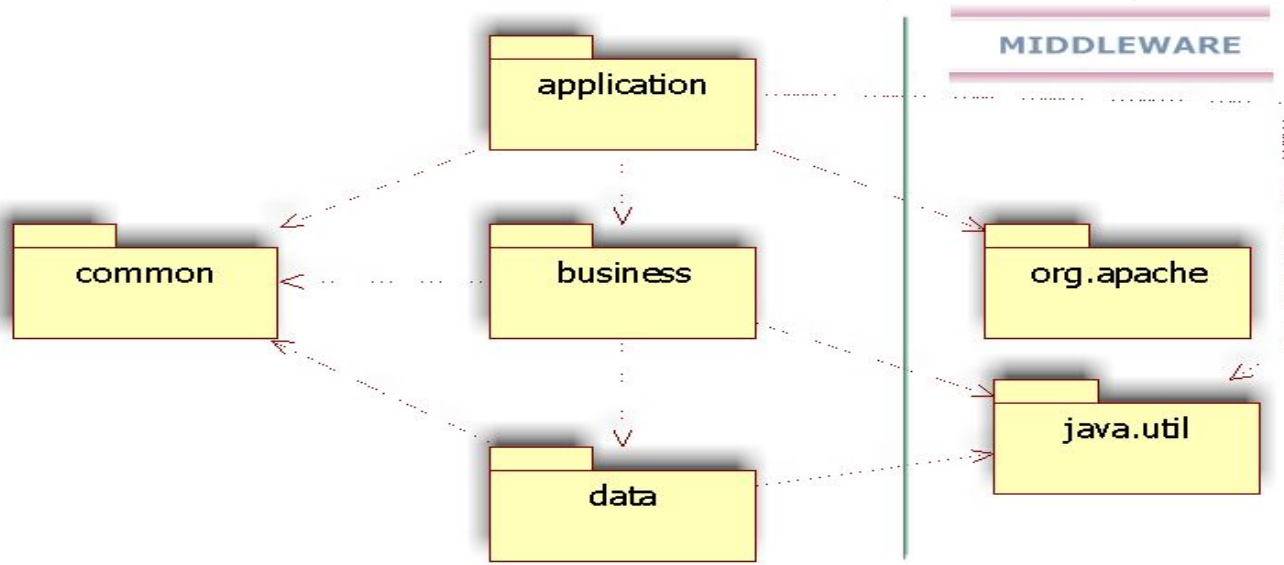


Tomado de Designing Data Intensive Web Applications. Stefano et al.



Representación de las capas

- Cada paquete es un cluster de objetos colaborativos que dependen de las facilidades ofrecidas por las capas más bajas



CASO DE ESTUDIO: BACKEND



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes Regionales

Funcionalidad para “Insertar una película”

Insertar un nueva película

Título:

Género:

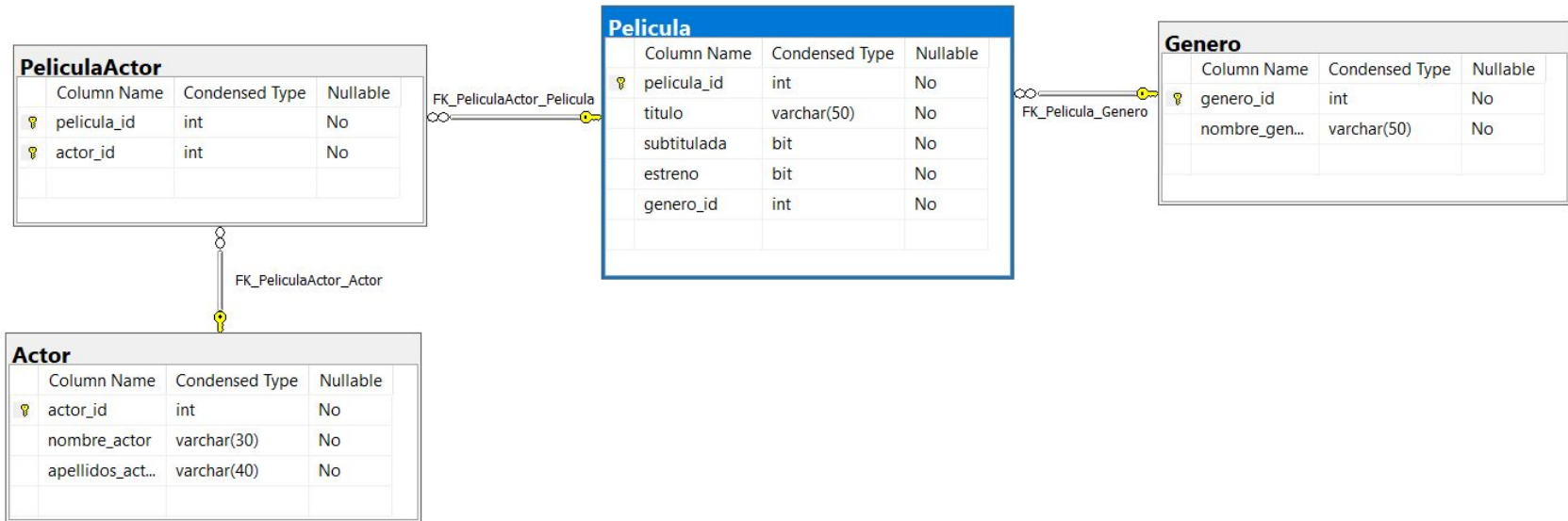
Total Películas:

☒ Estreno

☒ Subtitulada



E-R VideoRent



Paquete Común para todas las capas

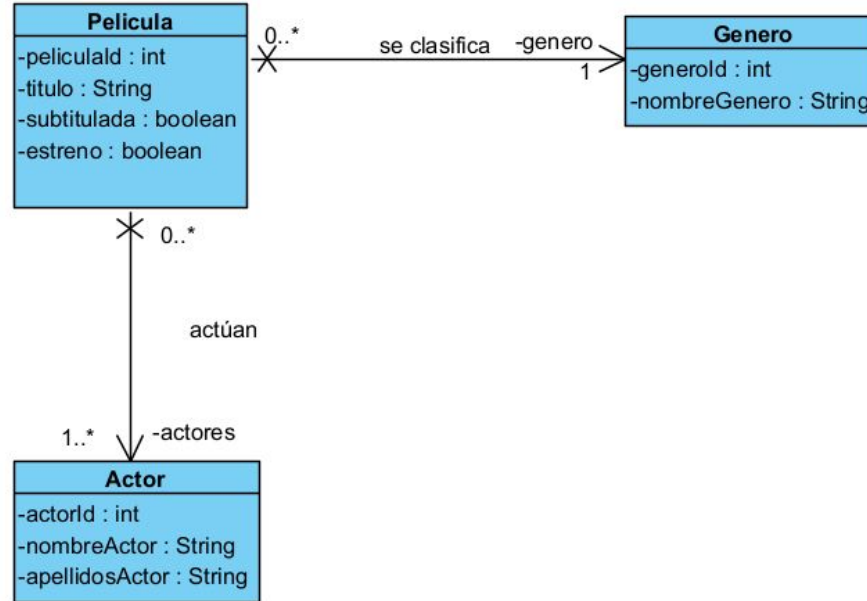
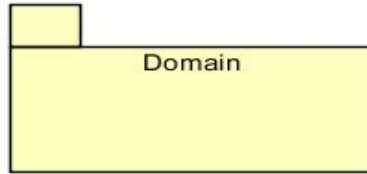
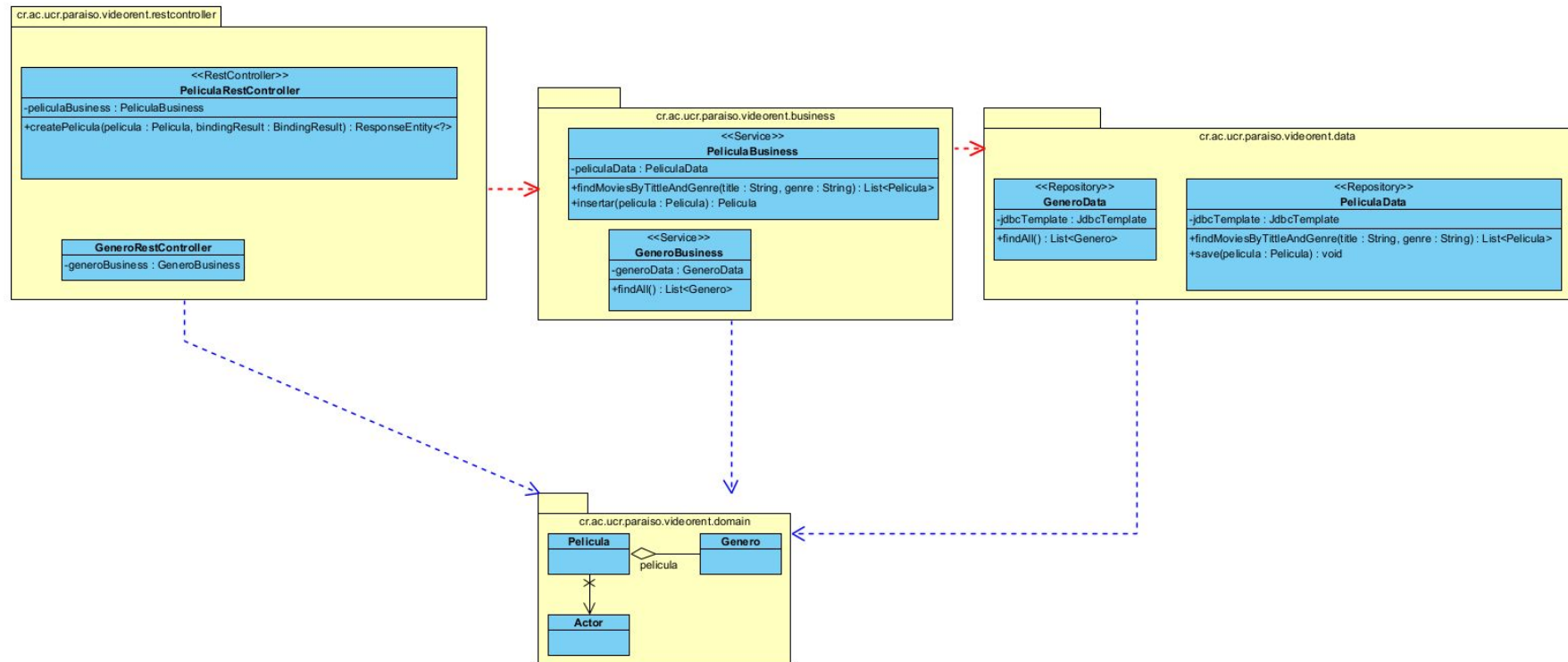
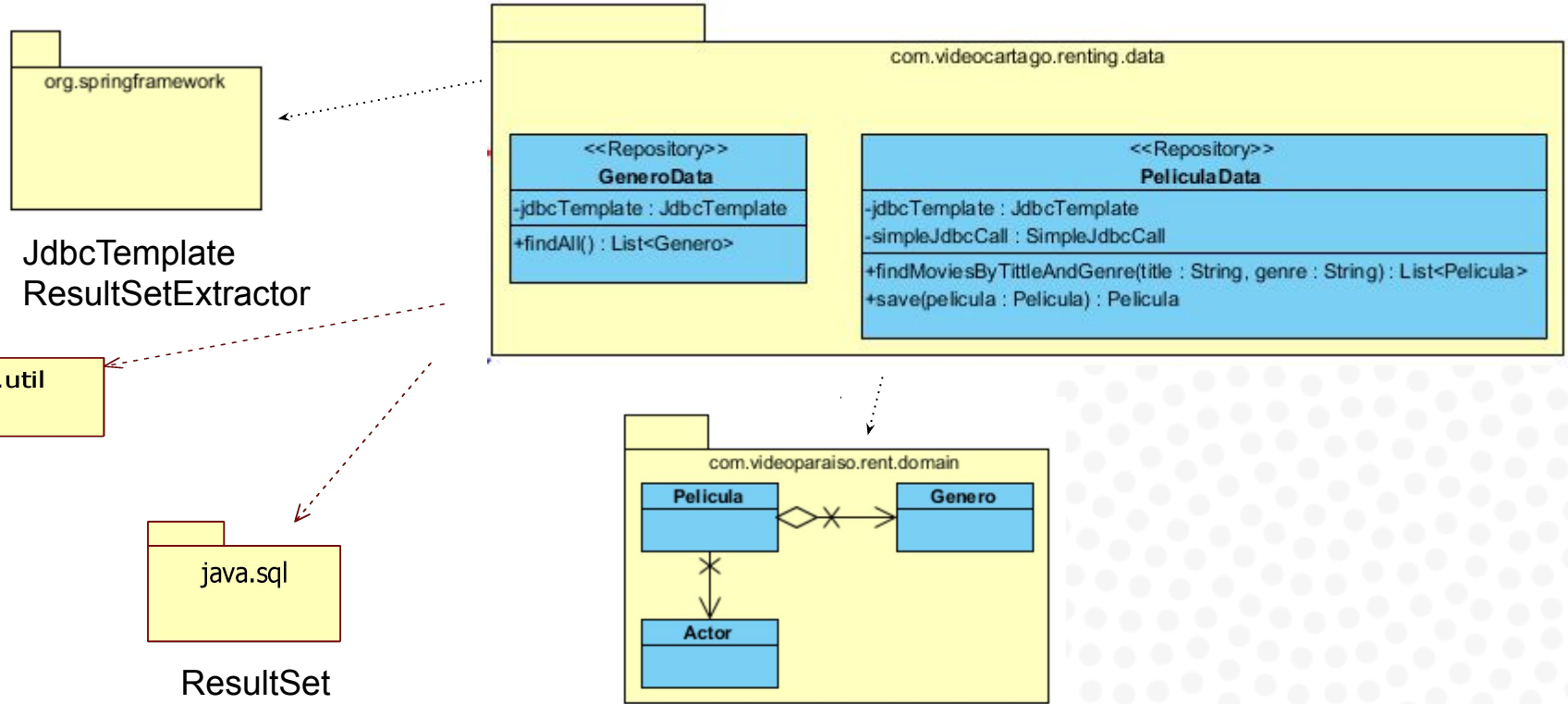


Diagrama de paquetes - CAPAS



Capa de Lógica de Acceso a Datos



Capa de Lógica de Aplicación:RestController

Insertar un nueva película

Título:

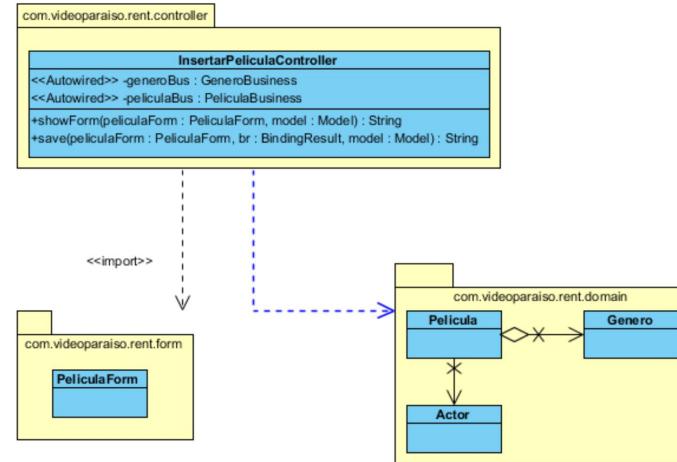
Género:

Total Películas:

☒ Estreno ☒ Subtitulada

insertarPelicula.html

java.util



La **instanciación** de un framework se realiza a través de la **composición** y **extensión** de las clases existentes.



Código del backend requerido



1. Código de la Clase GeneroData

```
package cr.ac.ucr.paraíso.videorent.data;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;
import cr.ac.ucr.paraíso.videorent.domain.Genero;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
@Repository
public class GeneroData {
    @Autowired
    private JdbcTemplate jdbcTemplate;
    public List<Genero> findAll() {
        String selectSql = "SELECT genero_id, nombre_genero FROM Genero";
        List<Genero> generos = jdbcTemplate.query(selectSql, new GeneroRowMapper());
        return generos;
    }
    private static final class GeneroRowMapper implements RowMapper<Genero> {
        @Override
        public Genero mapRow(ResultSet rs, int rowNum) throws SQLException {
            return new Genero(rs.getInt("genero_id"), rs.getString("nombre_genero"));
        }
    }
}
```

order by?



1. Código de la Clase GeneroDataTest (Prueba unitaria)

```
USE videorent;
-- Delete all records from required tables
DELETE FROM PeliculaActor;
DELETE FROM Pelicula;
DELETE FROM Actor;
DELETE FROM Genero;
-- Insert new genero
-- Insert data into Genero table
INSERT INTO Genero (nombre_genero) VALUES
('Action'),
('Comedy'),
('Drama'),
('Sci-Fi'),
('Thriller'),
('Horror'),
('Animation'),
('Adventure'),
('Romance'),
('Crime');
```

```
import org.springframework.test.context.jdbc.Sql;
import org.springframework.test.context.jdbc.Sql.ExecutionPhase;
import cr.ac.ucr.paraíso.videorent.domain.Genero;
@SpringBootTest
class GeneroDataTest {
    @Autowired
    private GeneroData generoData;
    @Test
    @Sql (scripts = "/findAll_recupera_todos_los_generos.sql",
        executionPhase = ExecutionPhase.BEFORE_TEST_METHOD)
    void findAll_recupera_todos_los_generos() {
        //Act
        List<Genero> generos = generoData.findAll();
        int cantidadRegistrosRetornados = generos.size();
        //Arrange
        int cantidadRegistrosEsperados = 10;
        assertEquals(cantidadRegistrosEsperados,
            cantidadRegistrosRetornados);
    }
}
```



¿Por qué necesitamos pruebas automatizadas?

Detección temprana de errores

Identifica fallos en las primeras etapas del ciclo de desarrollo, reduciendo costos de reparación.

Calidad y diseño del código

Fomenta mejores prácticas de desarrollo y un diseño de software más limpio y modular.

Refactorización segura

Permite mejorar el código existente con la garantía de que la funcionalidad no se verá afectada.

Reducción de errores de regresión

Asegura que los nuevos cambios no rompan las funcionalidades que ya funcionaban correctamente.

Soporte para tuberías CI/CD

Facilita la integración y entrega continua mediante validaciones automáticas constantes.

Confianza en el despliegue

Aumenta la seguridad del equipo al liberar cambios a producción con mayor frecuencia.



Test Pyramid in Spring



04

End-to-End Tests

Full application flow (less common in Spring).

03

Web / API Tests

Controllers and REST endpoints.

02

Integration Tests

Load Spring context; test interaction between components.

01

Unit Tests

Test individual classes or methods; no Spring context.

2. Creación procedimiento almacenado

```
CREATE PROCEDURE Pelicula_Insert(  
@pelicula_id int output,  
@titulo varchar(max),  
@subtitulada bit,  
@estreno bit,  
@generoId int  
)  
AS  
BEGIN  
    INSERT INTO dbo.Pelicula (titulo,subtitulada,estreno, genero_id)  
    VALUES(RTRIM(@titulo),@subtitulada,@estreno, @generoId);  
    SELECT @pelicula_id = @@IDENTITY  
END
```



2. Creación procedimiento almacenado

```
CREATE PROCEDURE PeliculaActor_Insert
    @pelicula_id int,
    @actor_id int
AS
BEGIN
    INSERT INTO dbo.PeliculaActor (pelicula_id, actor_id)
    VALUES (@pelicula_id, @actor_id);
END
```



3. Implementación de la clase PeliculaData

```
25 @Repository
26 public class PeliculaData {
27
28     @Autowired
29     private JdbcTemplate jdbcTemplate;
30
31     // The @Transactional annotation bounds the method execution in a transaction context
32     // as it is performing a database update.
33     @Transactional
34     public void save(Pelicula pelicula) throws SQLException{
35         SimpleJdbcCall simpleJdbcCallPelicula = new SimpleJdbcCall(jdbcTemplate).
36             withCatalogName("dbo").
37             withProcedureName("InsertPelicula").withoutProcedureColumnMetaDataAccess().
38             declareParameters(new SqlOutParameter("@pelicula_id", Types.INTEGER)).
39             declareParameters(new SqlParameter("@titulo", Types.VARCHAR)).
40             declareParameters(new SqlParameter("@subtitulada", Types.BIT)).
41             declareParameters(new SqlParameter("@estreno", Types.BIT)).
42             declareParameters(new SqlParameter("@genero_id", Types.INTEGER));
43         Map<String, Object> outParameters = simpleJdbcCallPelicula.execute(pelicula.getTitulo(),
44             pelicula.isSubtitulada(), pelicula.isEstreno(), pelicula.getGenero().getGeneroId());
45         pelicula.setPeliculaId(Integer.parseInt(outParameters.get("@pelicula_id").toString()));
46
47         SimpleJdbcCall simpleJdbcCallPeliculaActor = new SimpleJdbcCall(jdbcTemplate).
48             withCatalogName("dbo").
49             withProcedureName("InsertPeliculaActor").withoutProcedureColumnMetaDataAccess().
50             declareParameters(new SqlParameter("@pelicula_id", Types.INTEGER)).
51             declareParameters(new SqlParameter("@actor_id", Types.INTEGER));
52         for(Actor actor: pelicula.getActores())
53             simpleJdbcCallPeliculaActor.execute(pelicula.getPeliculaId(), actor.getActorId());
54     }
```



3. Implementación de la clase PelículaDataTest (Prueba Unitaria)

```
USE videorent;
-- Delete all records from required tables
DELETE FROM PeliculaActor;
DELETE FROM Pelicula;
DELETE FROM Actor;
DELETE FROM Genero;
-- Insert new genero
INSERT INTO Genero (nombre_genero) VALUES ('Action');
-- Capture generated genero_id
DECLARE @genero_id INT = SCOPE_IDENTITY();
-- Insert new actors
INSERT INTO Actor (nombre_actor, apellidos_actor) VALUES
('Keanu', 'Reeves');
DECLARE @actor1_id INT = SCOPE_IDENTITY();
INSERT INTO Actor (nombre_actor, apellidos_actor) VALUES
('Laurence', 'Fishburne');
DECLARE @actor2_id INT = SCOPE_IDENTITY();
```



3. Implementación de la clase PelículaDataTest (Prueba unitaria)

```
@SpringBootTest
public class PelículaDataTest {
    @Autowired
    private PelículaData películaData;

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Test
    @Sql (scripts = "/save_película_con_actores.sql", executionPhase = ExecutionPhase.BEFORE_TEST_METHOD)
    public void save_película_con_actores() {
        // Assert MEJORADO CON UNA INYECCIÓN DE REGISTROS PARA MEJORAR LA PRUEBA UNITARIA
        // IDENT_CURRENT() returns the last identity value generated for a specific table, regardless of the scope.
        // Retrieve generated genero_id
        Integer generoId = jdbcTemplate.queryForObject("SELECT IDENT_CURRENT('Genero')", Integer.class);

        // Retrieve generated actor1_id and actor2_id
        Integer actorId1 = jdbcTemplate.queryForObject("SELECT IDENT_CURRENT('Actor')-1", Integer.class);
        Integer actorId2 = jdbcTemplate.queryForObject("SELECT IDENT_CURRENT('Actor')", Integer.class);

        List<Actor> actores = new LinkedList<>();
        actores.add(new Actor(actorId1, null, null));
        actores.add(new Actor(actorId2, null, null));
        Película película = new Película(0, "The Matrix", true, true, new Genero(generoId, null), actores);

        //Act
        assertDoesNotThrow(() -> películaData.save(película));
        // Assert
        assertEquals(0, película.getId());
    }
}
```



5. Implementación de la clase GeneroBusiness

```
package com.videcartago.renting.business;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.videcartago.renting.data.GeneroData;
import com.videcartago.renting.domain.Genero;

@Service
public class GeneroBusiness {
    @Autowired
    private GeneroData generoData;

    public List<Genero> findAll() {
        return generoData.findAllGenres();
    }
}
```



4. Implementación de la clase PeliculaBusiness

```
package com.videcartago.renting.business;

import java.sql.SQLException;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.videcartago.renting.data.PeliculaData;
import com.videcartago.renting.domain.Pelicula;

@Service
public class PeliculaBusiness {

    @Autowired
    private PeliculaData peliculaData;

    public List<Pelicula> findAllMoviesByTitleAndGenre(String title, String genre) {
        return peliculaData.findMoviesByTitleAndGenre(title, genre);
    }

    public Pelicula save(Pelicula pelicula) throws SQLException{
        return peliculaData.save(pelicula);
    }
}
```



Creación de una clase Data Transfer Object

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```
import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
public class PeliculaCreationDTO {
    // I'm removing peliculaId for creation, as it's
    // often auto-generated by the DB
    @NotBlank(message = "Title is mandatory")
    @Size(min = 1, max = 50, message = "Title must be
between 1 and 50 characters")
    private String titulo;
    private boolean subtitulada;
    private boolean estreno;
    @NotNull(message = "Genre is mandatory")
    @Valid // Important: Validates the nested GeneroDTO
    private GeneroDTO genero;
    @NotNull(message = "Actors are mandatory")
    @Size(min = 1, message = "At least one actor is
required")
    @Valid // Important: Validates the nested ActorDTO
    List
    private List<ActorDTO> actores;
```

Creación de una clase Data Transfer Object

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

```
// Nested DTOs (Important!)
public static class GeneroDTO {
    @NotNull(message = "Genre ID is mandatory")
    private int generoId;
    public int getGeneroId() {
        return generoId;
    }
    public void setGeneroId(int generoId) {
        this.generoId = generoId;
    }
}

public static class ActorDTO {
    @NotNull(message = "Actor ID is mandatory")
    private int actorId;
    public int getActorId() {
        return actorId;
    }
    public void setActorId(int actorId) {
        this.actorId = actorId;
    }
}
}
```



Creación de una clase Mapper

```
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>1.5.5.Final</version>
</dependency>

<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct-processor</artifactId>
  <version>1.5.5.Final</version>
  <scope>provided</scope>
</dependency>
```

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper(componentModel = "spring")
public interface PeliculaMapper {

    @Mapping(source = "genero.generoId", target = "genero.generoId")
    @Mapping(target = "actores", expression =
"java(mapActors(peliculaCreationDTO.getActores()))")
    Pelicula toPelicula(PeliculaCreationDTO peliculaCreationDTO);

    default List<Actor> mapActors(List<PeliculaCreationDTO.ActorDTO>
actorDTOs) {
        if (actorDTOs == null) {
            return null;
        }
        return actorDTOs.stream()
            .map(actorDTO -> {
                Actor actor = new Actor();
                actor.setActorId(actorDTO.getActorId());
                return actor;
            }).collect(java.util.stream.Collectors.toList());
    }

    default Genero mapGenero(PeliculaCreationDTO.GeneroDTO generoDTO){
        if(generoDTO == null){
            return null;
        }
        Genero genero = new Genero();
        genero.setGeneroId(generoDTO.getGeneroId());
        return genero;
    }
}
```

Creación de la clase PeliculaRestController

```
@RestController
@RequestMapping(value = "/peliculas")
public class PeliculaRestController {
    @Autowired
    private PeliculaBusiness peliculaBusiness;

    @Autowired
    private PeliculaMapper peliculaMapper; // Inject the mapper

    @PostMapping
    public ResponseEntity<?> createPelicula(@Validated @RequestBody PeliculaCreationDTO peliculaDTO,
        BindingResult bindingResult) {
        if (bindingResult.hasErrors()) {
            List<FieldError> errors = bindingResult.getFieldErrors();
            List<String> errorMessages = errors.stream()
                .map(error -> error.getField() + ": " + error.getDefaultMessage())
                .collect(Collectors.toList());
            return ResponseEntity.badRequest().body(errorMessages);
        }
        Pelicula pelicula = peliculaMapper.toPelicula(peliculaDTO);
        try {
            this.peliculaBusiness.save(pelicula);
        } catch (SQLException e) {
            // TODO what to do with this
            e.printStackTrace();
        }

        return new ResponseEntity<>(pelicula, HttpStatus.CREATED);
    }
}
```



Uso del Postman

1. Abra Postman:

2. Cree un nuevo Request:

- Haga clic en el botón "New" en la esquina superior izquierda
- Seleccione "HTTP Request".

3. Configure el Request:

- **HTTP Method:** Seleccione "POST" del menú desplegable.
- **Ingrese la URL:** localhost:8080/videorent/peliculas

Configure los encabezados:

- Vaya a la pestaña "Headers".
- Agregue el siguiente encabezado:
 - Key: Content-Type
 - Value: application/json

Configure el cuerpo del Request:

- Vaya a la pestaña "Body".
- Seleccione "raw" y luego "JSON" del desplegable.
- Ingrese los datos JSON del objeto Pelicula :

```
{
  "titulo": "The Godfather",
  "subtitulada": false,
  "estreno": false,
  "genero": {
    "generoId": 1
  },
  "actores": [
    {
      "actorId": 1045
    },
    {
      "actorId": 1046
    }
  ]
}
```



Uso del Postman

The screenshot displays the Postman application interface. On the left, the 'My Workspace' sidebar shows a collection named 'Película' with four items: 'GET Get data', 'POST Post data' (selected), 'PUT Update data', and 'DEL Delete data'. The main workspace shows a 'POST' request to 'http://localhost:8080/videoent/peliculas'. The 'Body' tab is active, showing a JSON payload. The response is a '201 Created' status with a 70 ms response time and 422 B of data. The response body is also shown in JSON format.

Request Details:

- Method: POST
- URL: http://localhost:8080/videoent/peliculas
- Body Type: raw (JSON)

Request Body (JSON):

```
1 {
2   "películaId": 1976,
3   "titulo": "The Godfather",
4   "subtitulada": false,
5   "estreno": false,
6   "genero": {
7     "generoId": 52,
8     "nombreGenero": null
9   },
10  "actores": [
11    {
12      "actorId": 1045,
13      "nombreActor": null,
14      "apellidosActor": null
15    },
16    {
17      "actorId": 1046,
18      "nombreActor": null,
19      "apellidosActor": null
20    }
21  ]
22 }
```

Response Details:

- Status: 201 Created
- Time: 70 ms
- Size: 422 B

Response Body (JSON):

```
1 {
2   "mensaje": "Se creó correctamente la película."
3 }
```



Creación de RestController para Genero



UNIVERSIDAD DE
COSTA RICA

SR-CIE

Carrera de
Informática Empresarial
Sedes Regionales

Creación de la clase GeneroDTO

```
package cr.ac.ucr.paraíso.videorent.dto;
public class GeneroDTO {
    private int id;
    private String nombre;
    // Constructors
    public GeneroDTO() {}
    public GeneroDTO(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }
    // Getters and Setters

}
```



Creación de la clase GeneroRestController

```
@RestController
@RequestMapping(value = "/generos")
@CrossOrigin(origins = "http://localhost:4200")
public class GeneroRestController {
    @Autowired
    private GeneroBusiness generoBusiness;

    @GetMapping
    public ResponseEntity<List<GeneroDTO>> getAllGeneros() {
        List<Genero> generos = generoBusiness.findAll();

        if (generos.isEmpty()) {
            return ResponseEntity.noContent().build();
        }
        // Map Genero to GeneroDTO
        List<GeneroDTO> generoDTOS =
            generos.stream().map(this::convertToDTO).collect(Collectors.toList());

        return ResponseEntity.ok(generoDTOS);
    }
    private GeneroDTO convertToDTO(Genero genero) {
        return new GeneroDTO(genero.getGeneroId(), genero.getNombreGenero());
    }
}
```



Creación de RestController para Actor



Creación de la clase ActorData

```
@Repository
public class ActorData {
    @Autowired
    private JdbcTemplate jdbcTemplate;
    public List<Actor> findAll() {
        String selectSql = "SELECT actor_id, nombre_actor,
        apellidos_actor FROM Actor";
        List<Actor> actores = jdbcTemplate.query(selectSql,
        new ActorRowMapper());
        return actores;
    }
    private static final class ActorRowMapper implements
    RowMapper<Actor> {
        @Override
        public Actor mapRow(ResultSet rs, int rowNum) throws
        SQLException {
            return new Actor(rs.getInt("actor_id"),
            rs.getString("nombre_actor"),
            rs.getString("apellidos_actor"));
        }
    }
}
```



Creación de la clase ActorDTO

```
public class ActorDTO {  
    private int id;  
    private String nombre;  
    private String apellidos;  
  
    public ActorDTO() {  
        // TODO Auto-generated constructor stub  
    }  
    public ActorDTO(int id, String nombre,  
        String apellidos) {  
        super();  
        this.id = id;  
        this.nombre = nombre;  
        this.apellidos = apellidos;  
    }  
    // métodos set y get  
}
```



Creación de la clase ActorRestController

```
@RestController
@RequestMapping(value = "/actores")
@CrossOrigin(origins = "http://localhost:4200")
public class ActorRestController {
    @Autowired
    private ActorBusiness actorBusiness;
    @GetMapping
    public ResponseEntity<List<ActorDTO>> getAllGeneros() {
        List<Actor> actores = actorBusiness.findAll();
        if (actores.isEmpty()) {
            return ResponseEntity.noContent().build();
        }
        // Map Actor to ActorDTO
        List<ActorDTO> actorDTOS =
        actores.stream().map(this::convertToDTO).
        collect(Collectors.toList());
        return ResponseEntity.ok(actorDTOS);
    }
    private ActorDTO convertToDTO(Actor actor) {
        return new ActorDTO(actor.getActorId(),
        actor.getNombreActor(), actor.getApellidosActor());
    }
}
```

Referencias

Baeldung. (2025). *The DTO Pattern (Data Transfer Object)* | Baeldung. Baeldung.
<https://www.baeldung.com/java-dto-pattern>

Lhotka, R. (2009). *Expert C# 2008 Business Objects*. USA:Apress – Cap. 1 Distributed Architecture (lectura)

Sarknas, P. (2006). *Pro ASP.NET 2.0 E-Commerce in C#*. USA:Apress. Cap 10

Connolly, R. (2007). *Core Web application development with ASP .NET 2.0*. USA:Pearson Education – Cap. 11 Designing and Implementing Web Applications

